Projet d'évaluation

# Un éditeur de documents partagé

Développement web et cloud

Vous allez concevoir, construire et déployer sur AWS une application web d'écriture à plusieurs, puis prouver qu'elle tient les contraintes ci-dessous. Le sujet décrit ce que l'application doit faire et les conditions dans lesquelles elle doit le faire, pas la façon de la construire : le choix de l'architecture, des services et des technologies vous revient, et c'est l'essentiel de ce qui sera évalué.

## 1. L'application

Une association étudiante publie chaque année un journal numérique, écrit en commun par plusieurs centaines de rédacteurs. Les rédacteurs travaillent à plusieurs sur les mêmes textes, parfois au même endroit. À la parution, l'association diffuse la production par un lien de téléchargement, annoncé à une date et une heure précises, et limité à 500 exemplaires. L'équipe continue de corriger après l'annonce : une partie corrigée après la parution doit se retrouver dans les exemplaires téléchargés ensuite, sans que personne ait à changer de lien.

Un rédacteur doit pouvoir :

— créer un document (un simple texte) et y donner accès à d'autres rédacteurs ;

— écrire à plusieurs en même temps dans un même document, chacun voyant presque aussitôt les modifications des autres, et sachant qui est présent et où il écrit ;

— consulter les versions précédentes d'un document et en restaurer une ;

— créer, pour un document, un lien de téléchargement, en choisissant le nombre de téléchargements qu'il autorise.

Le lien ne permet de télécharger que ce nombre de fois, puis il l'indique clairement. Il donne toujours le document tel qu'il est au moment où l'on s'en sert, et non tel qu'il était à la création du lien. Le fichier téléchargé est un PDF.

Chaque rédacteur dispose d'un crédit d'écriture de 10 000 caractères, rechargé chaque jour. Quand son crédit est épuisé, il ne peut plus écrire avant la prochaine recharge. Une personne qui s'acharne contre ces limites est repérée par l'application, et bloquée pour un temps.

L'association doit pouvoir consulter à tout moment l'état de l'application : les documents ouverts, les liens actifs avec leur nombre de téléchargements restants, et les personnes bloquées.

Quand un lien arrive à son dernier téléchargement, son créateur en est prévenu par e-mail.

Il n'est pas demandé de gérer de vrais comptes : un nom et une adresse e-mail saisis à l'entrée suffisent à identifier un rédacteur. Les visiteurs qui téléchargent n'ont aucune identité à donner.

## 2. Les contraintes

Les deux premières contraintes sont les plus importantes : une application rapide qui ne les respecte pas sera considérée comme non fonctionnelle.

1. Un lien permet exactement le nombre de téléchargements autorisé à sa création : jamais un de plus, quel que soit le nombre de visiteurs qui cliquent au même instant.
2. Quel que soit le nombre de personnes qui écrivent en même temps, y compris au même endroit du texte, aucune modification acceptée n'est perdue, et tous les participants finissent par voir le même document.
3. Un lien créé à l'instant $T$ et utilisé à l'instant $T_1$, après une modification du document, donne le document tel qu'il est à $T_1$, complet et cohérent, sans qu'il soit nécessaire de créer un nouveau lien.
4. Un rédacteur n'écrit jamais plus que son crédit du jour, même s'il écrit depuis plusieurs appareils à la fois.
5. Le journal est annoncé à une heure précise : 15 000 personnes cliquent sur le lien, dont 80 % dans la première minute, pendant que 300 rédacteurs continuent d'écrire. L'application doit le supporter sans casser, ni jeter : pendant tout le pic, chaque visiteur obtient une réponse, et il sait s'il télécharge maintenant, s'il doit patienter ou si le lien est épuisé.
6. Une fois le dernier exemplaire distribué, les visiteurs encore présents l'apprennent rapidement, sans que l'application continue de délivrer des services qu'il ne dispose plus.

## 3. Les règles

— **Compte AWS.** Vous utilisez votre propre compte AWS, en plan gratuit. Toutes vos ressources sont dans une seule région.

— **Sécurité. Aucune clé d'accès AWS n'apparaît dans le code ou dans le dépôt.** Chaque composant n'a que les droits dont il a besoin.

— **Bonnes pratiques.** Vous appliquez les bonnes pratiques vues en cours.

— **Stockage.** Les fichiers à télécharger sont conservés dans S3.

— **Travail individuel.** Le projet est strictement personnel :

— **Délai.** Un week-end, du vendredi soir au lundi matin.

## 4. Ce que vous rendez

1. Un dépôt Git contenant le code de l'application, la description de l'infrastructure, les tests et un fichier README qui explique comment déployer, tester et supprimer l'ensemble.
2. Un dossier de conception.
3. Un rapport de test : la méthode, les objectifs que vous vous êtes fixés, les résultats bruts et leur analyse, et la preuve que les deux contraintes les plus importantes ont été respectées au cours de vos tests.
4. Une description de l'API de l'application, assez précise pour que le jury puisse écrire son propre test.

5. Une preuve de ce que le projet a coûté.

**Note :** Une application simple, mais dont chaque choix est compris et justifié, vaut mieux qu'une application complexe que vous ne savez pas expliquer.